Toggle switch for the login and registration sections, showing one and disabling the other when sign in or register are clicked.

```
if (loginSection && registerSection) {
    // toggle
    document.getElementById('show-register')?.addEventListener('click', e
=> {
        e.preventDefault();
        loginSection.classList.add('hidden');
        registerSection.classList.remove('hidden');
    });
    document.getElementById('show-login')?.addEventListener('click', e =>
{
        e.preventDefault();
        registerSection.classList.add('hidden');
        loginSection.classList.remove('hidden');
    });
```

Registration function for the form which is handled by an api call to the auth/register end point and validates via JWT.

```
 // registration
    document.getElementById('register-form').addEventListener('submit',
async function (e) {
        e.preventDefault();
        const name     = document.getElementById('reg-name').value.trim();
        const email    =
document.getElementById('reg-email').value.trim();
        const password =
document.getElementById('reg-password').value.trim();
        if (!name || !email || !password) { alert('Please fill in all
fields.'); return; }

        const { ok, status, data } = await apiFetch('/auth/register', {
            method: 'POST',
            body: JSON.stringify({ name, email, password })
        });

        if (ok) {
            localStorage.setItem('authToken', data.token);
            localStorage.setItem('loggedInUser', data.user.email);
```

Handler function for logging in. Parses API as a first resort and localStorage if that fails, depending on the machine setup.

```
            localStorage.setItem('registeredUser',
JSON.stringify(data.user));
            alert('Account created successfully!');
            window.location.href = 'index.html';
            return;
        }

        if (status === 0) {
            localStorage.setItem('registeredUser', JSON.stringify({ name,
email, password }));
            localStorage.setItem('loggedInUser', email);
            alert('Account created (offline mode)!');
            window.location.href = 'index.html';
            return;
        }

        if (status === 409) { alert('An account with that email already
exists.'); return; }
        alert(data.error || 'Registration failed.');
    });
}
```

Function for grabbing the vehicle alerts. Pulls the vehicle from the get vehicles function through the id, handles empty, and then gets the reminders to post on the services dashboard.

```
function getVehicleAlerts(vehicleId) {
    const vehicle = getVehicles().find(v => v.id === vehicleId);
    if (!vehicle) return [];
    const today = new Date();

    return getReminders().filter(r => r.vehicleId ===
vehicleId).reduce((alerts, r) => {
        const mode      = r.reminderMode || 'either';
        const usesMiles  = (mode === 'miles'  || mode === 'either') &&
Number(r.intervalMiles)  > 0;
        const usesMonths = (mode === 'time'   || mode === 'either') &&
Number(r.intervalMonths) > 0;
```

```
        const milesLeft = usesMiles
            ? Number(r.intervalMiles) - (Number(vehicle.mileage) -
Number(r.lastMileage))
            : null;

        const lastDate = new Date(r.lastDate);
        const monthsPassed = (today.getFullYear() -
lastDate.getFullYear()) * 12
            + (today.getMonth() - lastDate.getMonth());
        const monthsLeft = usesMonths ? Number(r.intervalMonths) -
monthsPassed : null;

        let overdue = false, dueSoon = false;
        if (mode === 'miles') {
            overdue  = usesMiles && milesLeft <= 0;
            dueSoon  = !overdue && usesMiles && milesLeft <= 5000;
        } else if (mode === 'time') {
            overdue  = usesMonths && monthsLeft <= 0;
            dueSoon  = !overdue && usesMonths && monthsLeft <= 6;
        } else {
            overdue  = (usesMiles && milesLeft <= 0) || (usesMonths &&
monthsLeft <= 0);
            dueSoon  = !overdue && ((usesMiles && milesLeft <= 5000) ||
(usesMonths && monthsLeft <= 6));
        }

        if (overdue) alerts.push({ type: 'overdue', service:
r.serviceType, milesLeft, monthsLeft });
        else if (dueSoon) alerts.push({ type: 'soon',    service:
r.serviceType, milesLeft, monthsLeft });
        return alerts;
    }, []);
}
```

Vehicle model open function to display data about the vehicle that’s being looked at.

```
    function openModal(vehicleId = null) {
        vehicleForm.reset();
        vehicleForm.dataset.editId = vehicleId || '';
        if (modalTitle) modalTitle.textContent = vehicleId ? 'Edit
Vehicle' : 'Add New Vehicle';
```

```
        if (vehicleId) {
            const v = getVehicles().find(v => v.id === vehicleId);
            if (v) {
                ['year','make','model','vin','plate','mileage'].forEach(f
=> {
                    const el = document.getElementById(`v-${f}`); if (el)
el.value = v[f] || '';
                });
                const fuel = document.getElementById('v-fuel');         if
(fuel) fuel.value = v.fuelType   || '';
                const oilT = document.getElementById('v-oil-type');     if
(oilT) oilT.value = v.oilType    || '';
                const oilI = document.getElementById('v-oil-interval'); if
(oilI) oilI.value = v.oilInterval|| '';
            }
        }
        modal.classList.remove('hidden');
    }
```

Page routing logic that checks which page the user is currently on so it knows which objects and views to display in the DOM, and also checks whether the user is logged in every time in order to be secure.

```
if      (page === 'garage.html')     { initGarage();     authBtnChange(); }
else if (page === 'log.html')        { initLog();         authBtnChange(); }
else if (page === 'index.html')      { initDashboard(); authBtnChange(); }
else if (page === 'reminders.html') { initReminders();  authBtnChange(); }
```

Module: Dashboard(index.html)

```
<script>
        const loggedInUser = localStorage.getItem('loggedInUser');

        if (!loggedInUser) {
            window.location.href = 'login.html';
        }
```

```
    </script>
```

Auth-Guard Logic

- Redirects authenticated users

```
<script>
        const loggedInEmail = localStorage.getItem('loggedInUser');
        const storedUser =
JSON.parse(localStorage.getItem('registeredUser'));

        const loginNav = document.getElementById('login-nav');
        const welcome = document.getElementById('welcomeUser');

        if (loggedInEmail) {

            // Default to email
            let displayName = loggedInEmail;

            // If registered user exists and matches, use name
            if (storedUser && storedUser.email === loggedInEmail) {
                displayName = storedUser.name;
            }

            // welcome message displaying users name
            if (welcome) {
                welcome.textContent = "Welcome Back, " + displayName +
"!";
            }

            // change login to logout
            if (loginNav) {
                loginNav.textContent = "Logout";
                loginNav.href = "#";
                loginNav.onclick = function(event) {
                    event.preventDefault();
                    localStorage.removeItem('loggedInUser');
                    window.location.href = 'login.html';
                };
            }
        }
```

```
    </script>
```

Session Management using local storage

- Keeps the user logged in after page reloads
- Greets with stored user name
- Converts login to logout
- Allows session termination

```
    <script src="script.js"></script>
```

- Data fetching logic is modularized to maintain separation of concerns between structure (HTML) and logic (JS)

Module: My Garage (garage.html)

```
<section class="vehicle-grid" id="vehicle-grid">
            <!--should be populated by the script-->
        </section>
```

Vehicle Grid

- Acts as a dynamic mounting point to be populated by the script
- Reflects users database records

```
<div id="vehicle-modal" class="modal-overlay hidden">
    <div class="modal-box">
        <h2 id="modal-title">Add New Vehicle</h2>
        <form id="vehicle-form">
            <div class="form-row">
                <div class="form-group">
                    <label for="v-year">Year <span
class="req">*</span></label>
                    <input type="number" id="v-year" min="1900" max="2100"
required>
                </div>
                <div class="form-group">
                    <label for="v-make">Make <span
class="req">*</span></label>
                    <input type="text" id="v-make" required>
                </div>
                <div class="form-group">
```

```
                    <label for="v-model">Model <span
class="req">*</span></label>
                    <input type="text" id="v-model" required>
                </div>
            </div>

            <div class="form-row">
                <div class="form-group">
                    <label for="v-vin">VIN</label>
                    <input type="text" id="v-vin" maxlength="17">
                </div>
            </div>

            <div class="form-group">
                <label for="v-mileage">Current Mileage <span
class="req">*</span></label>
                <input type="number" id="v-mileage" min="0" required>
            </div>
        </form>
    </div>
</div>
```

Form Architecture

- Takes input from the user to create an entry for a vehicle
- Uses required attributes for data integrity
- Uses range constraints to prevent logical errors for data sanitization in year
- Type constraints for validation

Module: Log Service(log.html)

```
<section class="log-box input-section">
    <h2>Record a Service</h2>
    <form id="maintenance-form">
        <div class="form-group">
            <label for="vehicle">Select Vehicle <span
class="req">*</span></label>
            <select id="vehicle" class="vehicle-select" required>
                <option value="">Choose a Vehicle</option>
                </select>
        </div>
```

```
        <div class="form-group">
            <label for="date">Date of Service <span
class="req">*</span></label>
            <input type="date" id="date" required>
        </div>

        <div class="form-group">
            <label for="mileage">Mileage at Service <span
class="req">*</span></label>
            <input type="number" id="mileage" placeholder="e.g. 45000"
min="0" required>
        </div>

        <button type="submit" class="btn">Log Service</button>
    </form>
</section>

<section class="log-box history-section">
    <h2>Service History</h2>
    <div class="filter-bar">
        <select id="filter-vehicle" class="vehicle-select">
            <option value="">All Vehicles</option>
        </select>
        <select id="filter-type">
            <option value="">All Services</option>
            <option value="Oil Change">Oil Change</option>
            </select>
    </div>
</section>
```

Input and Filtering

- Uses required attributes for data sanitization
- Uses input types for data sanitization
    - date-format confusion avoided
- History allows for two tiered filtering by vehicle and service type

Maintenance Reminder Dashboard Card

-Pulls all vehicles
-Calls getVehiclesAlerts() on vehicles and puts in array
-Sorts alerts by overdue vs due soon
-Puts alerts into a <li> element on card
-If no alerts are found for user, displays that all vehicles are up to date

```
// maintenance reminder card
if (reminderCard) {
    const vehicles = getVehicles();
    const alerts = [];

    reminderCard.innerHTML = '';

    vehicles.forEach(v => {
        const vReminders = getVehicleAlerts(v.id);
        vReminders.forEach(r => {
            alerts.push({ vehicle: v, service: r.service, milesLeft: r.milesLeft, monthsLeft: r.monthsLeft, type: r.type });
        });
    });

    if (alerts.length > 0) {
        alerts.sort((a, b) => {
            if (a.type === 'overdue' && b.type === 'soon') return -1;
            if (a.type === 'soon' && b.type === 'overdue') return 1;
            return 0;
        });

        alerts.forEach(alert => {
            const li = document.createElement('li');
            li.innerHTML =
                '<strong>' + alert.service + '</strong> for <strong>' +
                (alert.vehicle ? alert.vehicle.year + ' ' + alert.vehicle.make + ' ' + alert.vehicle.model : 'Unknown Vehicle') +
                '</strong> — ' + (alert.type === 'overdue' ? 'Overdue' : 'Due Soon') +
                (alert.type !== 'overdue' && alert.milesLeft > 0 ? ' • ' + alert.milesLeft + ' miles' : '') +
                (alert.type !== 'overdue' && alert.monthsLeft > 0 ? ' • ' + alert.monthsLeft + ' month' + (alert.monthsLeft !== 1 ? 's' : '') : '');
            reminderCard.appendChild(li);
        });
    } else {
        reminderCard.innerHTML = "<li>All vehicles are up to date! Great job!</li>";
    }
}
```

```
// recent activity card
if (recent) {
    const logs = getLogs();
    const vehicles = getVehicles();

    recent.innerHTML = '';

    if (logs.length > 0) {
        logs.sort((a, b) => new Date(b.date) - new Date(a.date));

        logs.slice(0, 5).forEach(log => {
            const v = vehicles.find(v => v.id === log.vehicleId);
            const li = document.createElement('li');

            li.innerHTML =
                '<strong>' + log.serviceType + '</strong> for <strong>' +
                (v ? v.year + ' ' + v.make + ' ' + v.model : 'Unknown Vehicle') +
                '</strong> on ' + log.date;

            recent.appendChild(li);
        });
    } else {
        recent.innerHTML = '<li>No recent activity.</li>';
    }
}
```

Recent Activity Card

- Pulls all logs
- Sorts them in a descending order by what date service was performed
- Users gets to view their 1-5 most recent service performed on cars in their garage
- If no cars are added, no services performed, display no recent activity

Code inspection Summary

During our inspection we went through each part of the tuneup application to make sure everything was working correctly and connected properly. We tested all the pages like the dashboard, my garage, log service, and about page to make sure navigation worked smoothly and nothing was broken

We also checked how the login system worked. The app correctly sends users back to the login page if they are not logged in. We made sure that once a user logs in, they stay logged in even after refreshing the page using localstorage. We also updated the navigation bar so it changes from “login” to “logout” when a user is signed in.

Bugs identified and fixes

While working on the project, we found and fixed a few issues:

- The navigation bar didn't change after logging in at first. We fixed this by making it switch from “Login” to “Logout” when the user is signed in
- The login session wasn't saving properly. We fixed this by correctly storing user data in localStorage
- Some pages links didn't work correctly at first, but we fixed them so all navigation works properly.

Testing results:
Users can register and log in, login stays active after refresh, users are redirected if not logged in, logout clears the session, navigation works across all pages, forms validate inputs correctly

Conclusion
The tuneup app meets the project requirements and works correctly. It provides a simple way to track vehicle maintenance. Users can register, log in, manage vehicles, and track maintenance without issues. The system provides a smooth and consistent user experience.